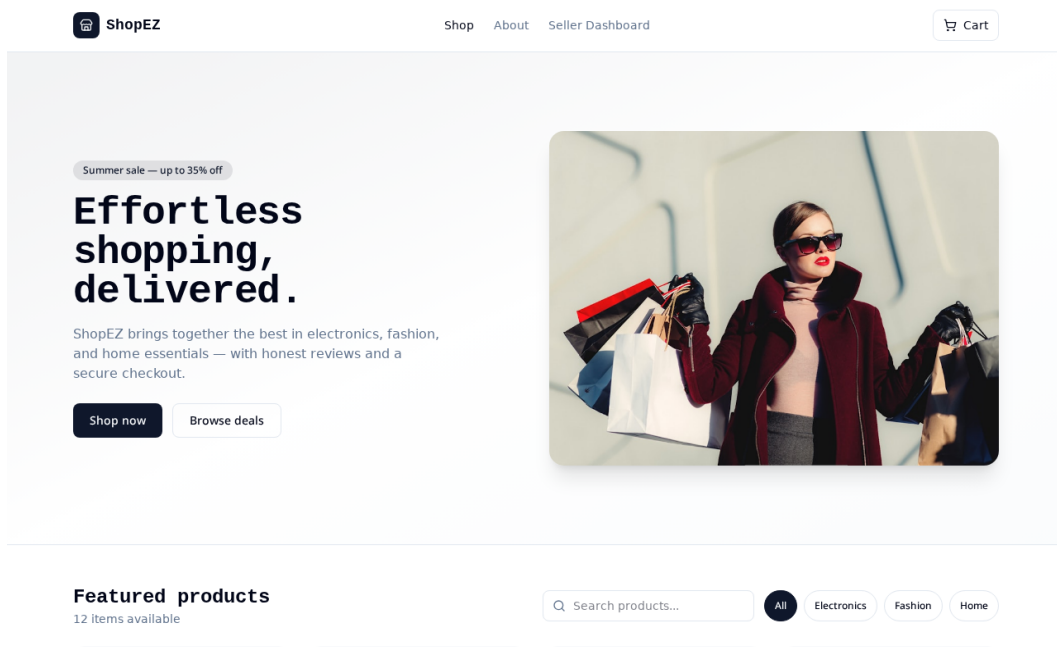


SHOPEZ

Full-Stack E-Commerce Web Application

Project Documentation



Author	Buyya Vaishnavi
Roll Number	24EG105R35
Institution	Anurag University
Version	1.0
Last Updated	30th June 2026

1. Introduction

Project Title: ShopEZ — Effortless Online Shopping Delivered.

ShopEZ is a full-stack e-commerce web application built with a modern React front-end and a TypeScript-based server layer. The store lets shoppers browse curated Electronics, Fashion, and Home products, add items to a persistent cart, and complete a demo checkout in a single flow. A separate Seller Dashboard gives merchants a live view of revenue, orders, growth and top-selling items.

2. Project Overview

Purpose

ShopEZ demonstrates a production-style commerce workflow — product discovery, cart, secure checkout and post-purchase confirmation — packaged in a clean, responsive interface. It also doubles as an admin analytics tool through the Seller Dashboard, making the same codebase useful for both shoppers and store operators.

Key Features

- User authentication with email login and personalised header greeting (e.g. “Hi, guru”)
- Product catalogue with category filters — All, Electronics, Fashion, Home
- Live product search across name, category and description
- Discount badges (–35%, –25%, –20%) and original-vs-sale price display
- Star ratings with review counts on every product card
- Shopping cart with quantity stepper, remove action and live order summary
- Multi-currency selector (INR  by default) in the top navigation
- Single-page checkout — shipping details and demo card payment side-by-side with order summary
- Order confirmation screen with unique Order ID (format: SHZ-XXXXXX)
- Seller Dashboard with Revenue, Orders, Products and Growth KPI cards
- Recent-orders table and Top-products list for merchants
- Fully responsive layout, monospace-styled accents and light theme

3. System Architecture

ShopEZ follows a classic three-tier architecture: a React presentation layer, a TypeScript server layer exposing typed server functions, and a document database for persistence.

3.1 Frontend Architecture (React + TanStack Router)

- **Router:** TanStack Router with file-based routes under `src/routes/`
- **Data layer:** TanStack Query for caching product lists and cart state
- **Styling:** Tailwind CSS v4 with a light, minimalist theme
- **State:** React Context for auth session, cart items and currency
- **Icons:** Lucide React (shop bag, cart, user, logout)

Key Pages

- Home / Shop — hero banner, featured products, category filter, search
- Cart — item list, quantity stepper, live order summary
- Checkout — shipping + demo payment form, place-order action
- Order Confirmed — success screen with generated Order ID
- Seller Dashboard — KPI cards, recent orders table, top products
- About — brand story and value propositions

3.2 Backend Architecture (TypeScript Server Functions)

- **Runtime:** Node.js 20 LTS on an edge-compatible serverless worker
- **RPC:** `createServerFn` for typed client→server calls
- **Auth:** JWT session tokens with bcrypt-hashed passwords
- **Validation:** Zod schemas on every server-function input
- **Public API:** Route handlers under `/api/public/*` for webhooks

3.3 Database (MongoDB)

- **users** — `_id`, `name`, `email`, `passwordHash`, `role`, `createdAt`
- **products** — `_id`, `title`, `category`, `price`, `originalPrice`, `discount`, `rating`, `reviews`, `image`, `stock`
- **orders** — `_id` (SHZ-XXXXXX), `userId`, `items[]`, `shipping`, `subtotal`, `tax`, `total`, `status`, `date`
- **carts** — `_id`, `userId`, `items[{productId, qty}]`, `updatedAt`

4. System Design Diagram

The diagram below shows the request path from the shopper’s browser through the React SPA, the typed server-function layer, and finally to MongoDB and any external service (payment / mail).

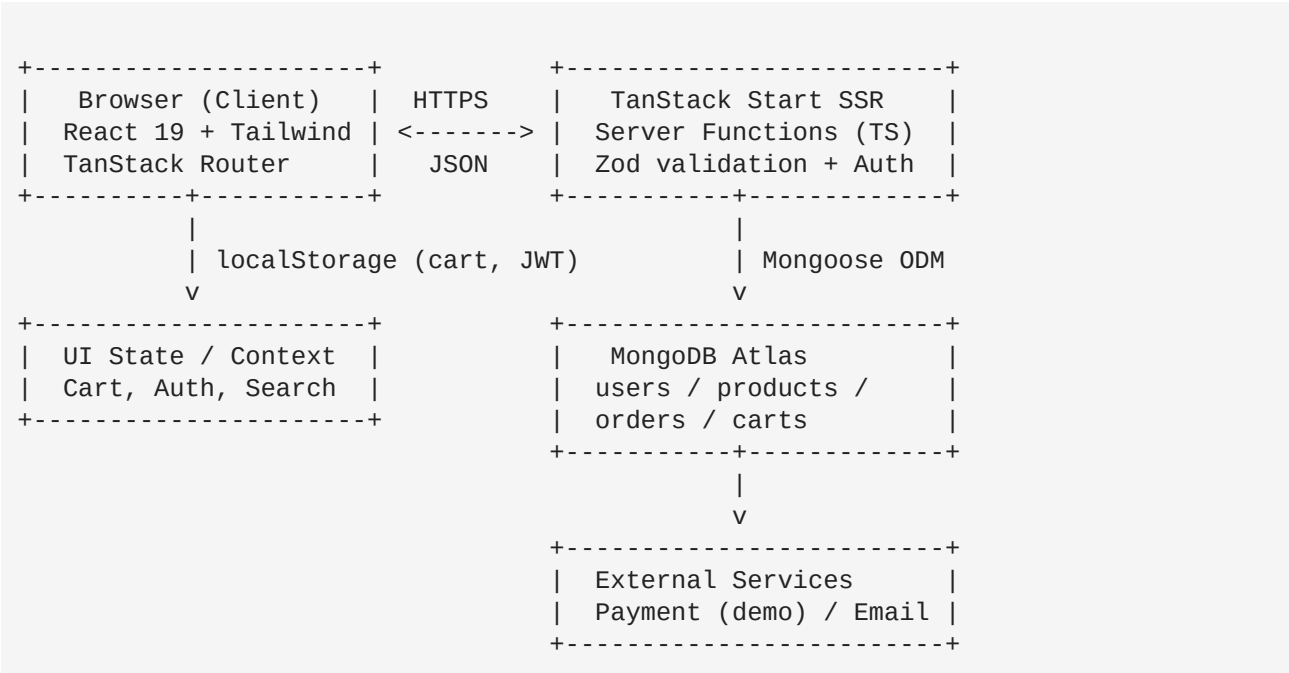


Figure 1. Three-tier request flow for ShopEZ.

5. User Flow — Purchase Flowchart

The flowchart traces a shopper from landing on the home page through a successful order confirmation.

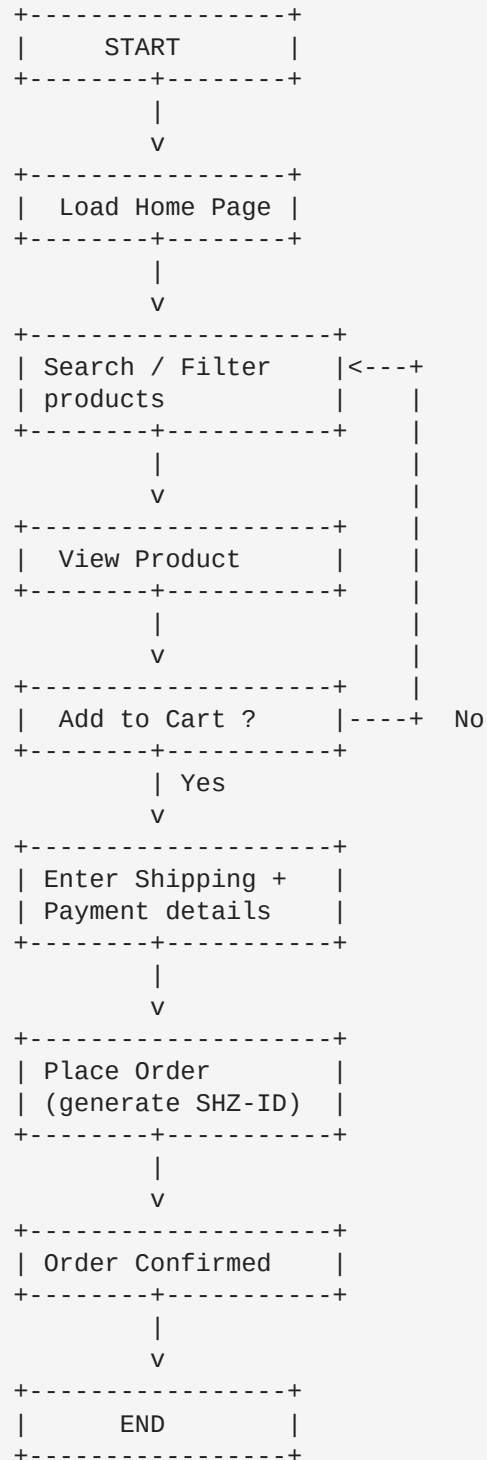


Figure 2. End-to-end shopper flow from browse to confirmation.

6. Setup Instructions

Prerequisites

- Node.js v20 LTS or newer
- npm, pnpm or bun package manager
- MongoDB Atlas account (or local MongoDB ≥ 6.0)
- Git and VS Code (recommended)

Step 1 — Clone

```
git clone https://github.com/buyya-vaishnavi/shopez.git
cd shopez
```

Step 2 — Install dependencies

```
bun install    # or: npm install
```

Step 3 — Environment variables

Create a `.env` file at the project root:

```
MONGO_URI=mongodb+srv://:@cluster.mongodb.net/shopez
JWT_SECRET=change-me-in-production
NODE_ENV=development
VITE_APP_NAME=ShopEZ
```

Step 4 — Run the dev server

```
bun run dev
```

The app is served at `http://localhost:8080`.

7. Folder Structure

```
shopez/  
  src/  
    routes/  
      __root.tsx      # root layout + head metadata  
      index.tsx       # Home / Shop  
      about.tsx  
      cart.tsx  
      checkout.tsx  
      order-confirmed.tsx  
      seller.tsx      # Seller Dashboard  
      api/public/  
        webhook.ts  
    components/  
      Navbar.tsx  
      ProductCard.tsx  
      CartItem.tsx  
      OrderSummary.tsx  
      Footer.tsx  
    context/  
      AuthContext.tsx  
      CartContext.tsx  
      CurrencyContext.tsx  
    lib/  
      products.functions.ts  
      orders.functions.ts  
      auth.server.ts  
    styles.css  
    router.tsx  
  public/  
  package.json  
  vite.config.ts  
  tsconfig.json
```

8. API & Server Functions

Authentication

- **signupFn(input)** — {name, email, password} → {user, token}
- **loginFn(input)** — {email, password} → {user, token}
- **logoutFn()** — clears session cookie

Products

- **listProductsFn(filter?)** — returns 12+ products with category filter
- **getProductFn(id)** — single product detail
- **searchProductsFn(query)** — full-text over title / category

Cart

- **getCartFn()** — returns items, subtotal, tax (8%), shipping, total
- **addToCartFn({productId, qty})**
- **updateQtyFn({productId, qty})**
- **removeFromCartFn({productId})**

Orders

- **placeOrderFn({shipping, payment})** → {orderId: "SHZ-XXXXXX", total, status}
- **myOrdersFn()** — authenticated shopper's history

Seller / Admin

- **sellerStatsFn()** — revenue, orders, products, growth %
- **recentOrdersFn(limit)**
- **topProductsFn(limit)**

9. Authentication & Security

Strategy

- JWT-based session tokens signed with `JWT_SECRET`
- Passwords hashed with `bcrypt` (10 rounds) before storage
- Tokens stored client-side in `localStorage` and sent via `Authorization` header
- Server functions marked with `.middleware([requireAuth])` reject anonymous calls

Roles

- **USER** — browse, cart, checkout, view own orders
- **SELLER / ADMIN** — access Seller Dashboard, edit products, view all orders

Session Flow

1. Shopper submits login form → 2. Server validates & issues JWT → 3. Header greets user (“Hi, guru”) → 4. Every protected call attaches the token → 5. Middleware verifies signature & user role.

10. User Interface — Home & Shop

The landing page opens with the “Effortless shopping, delivered.” hero, a summer-sale badge and quick-action buttons. Below the hero, the Featured Products grid lists 12 curated items with instant category filtering.

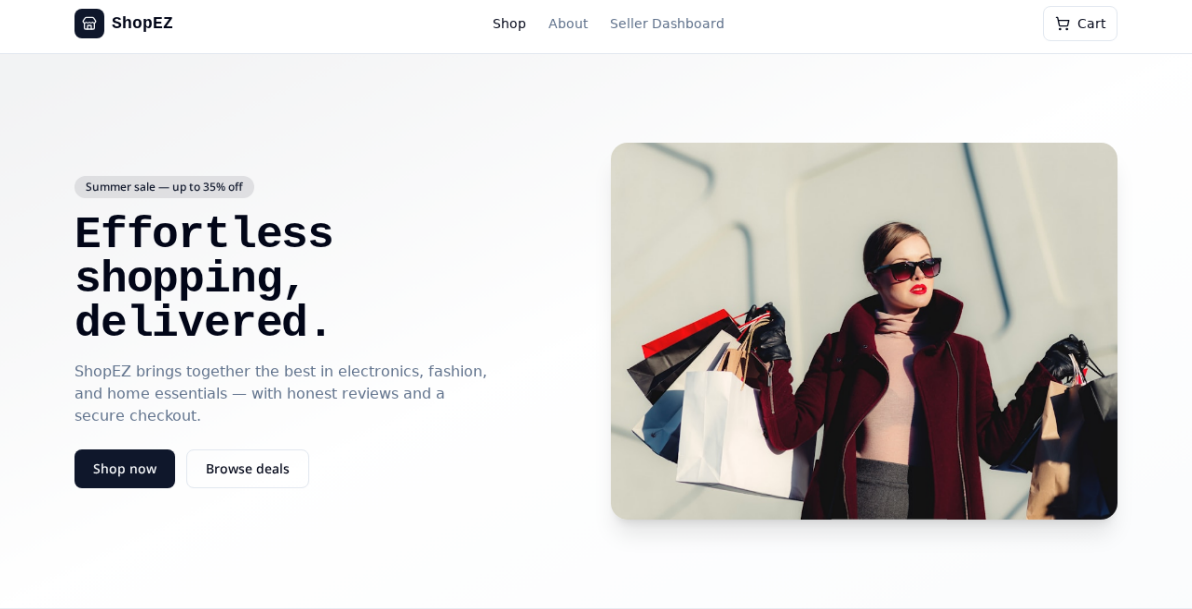


Figure 3. Home page — hero, sale badge and category filters.

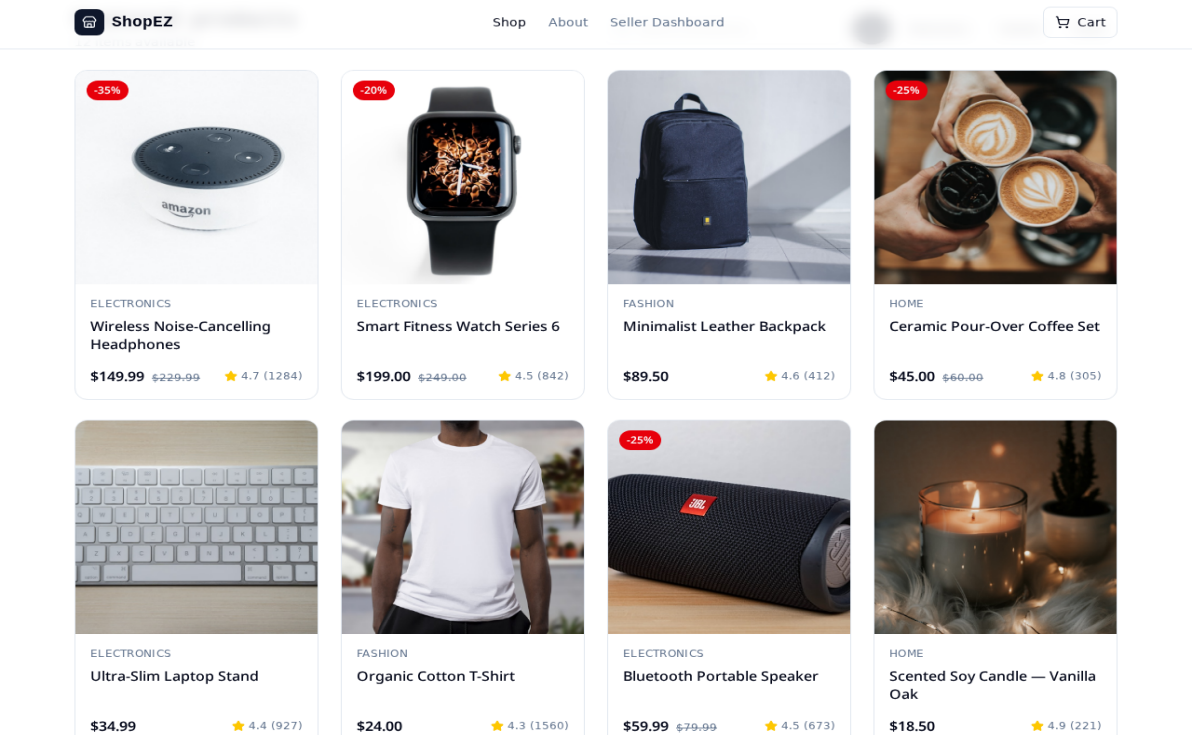


Figure 4. Featured Products grid with discount badges and ratings.

11. User Interface — Cart & Checkout

The cart page groups line items on the left with a sticky order summary on the right (subtotal, free shipping, 8% tax, total). Checkout keeps shipping and demo card details visible at the same time so the shopper can review the full order in one view before placing it.

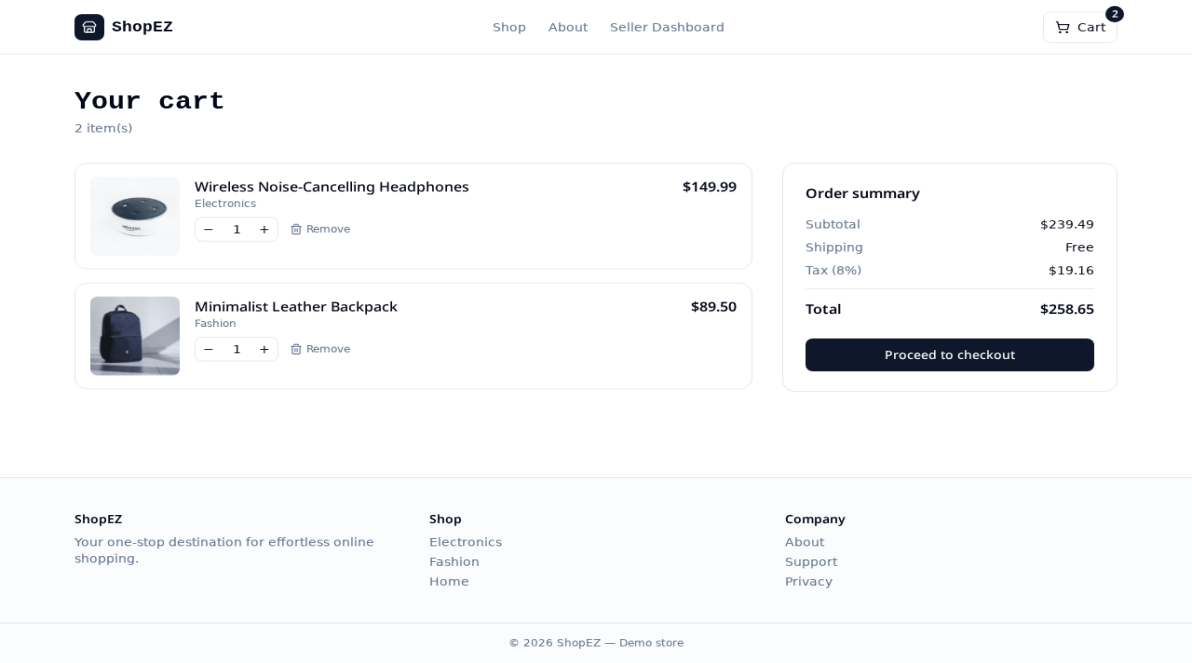


Figure 5. Cart page with quantity stepper and live totals.

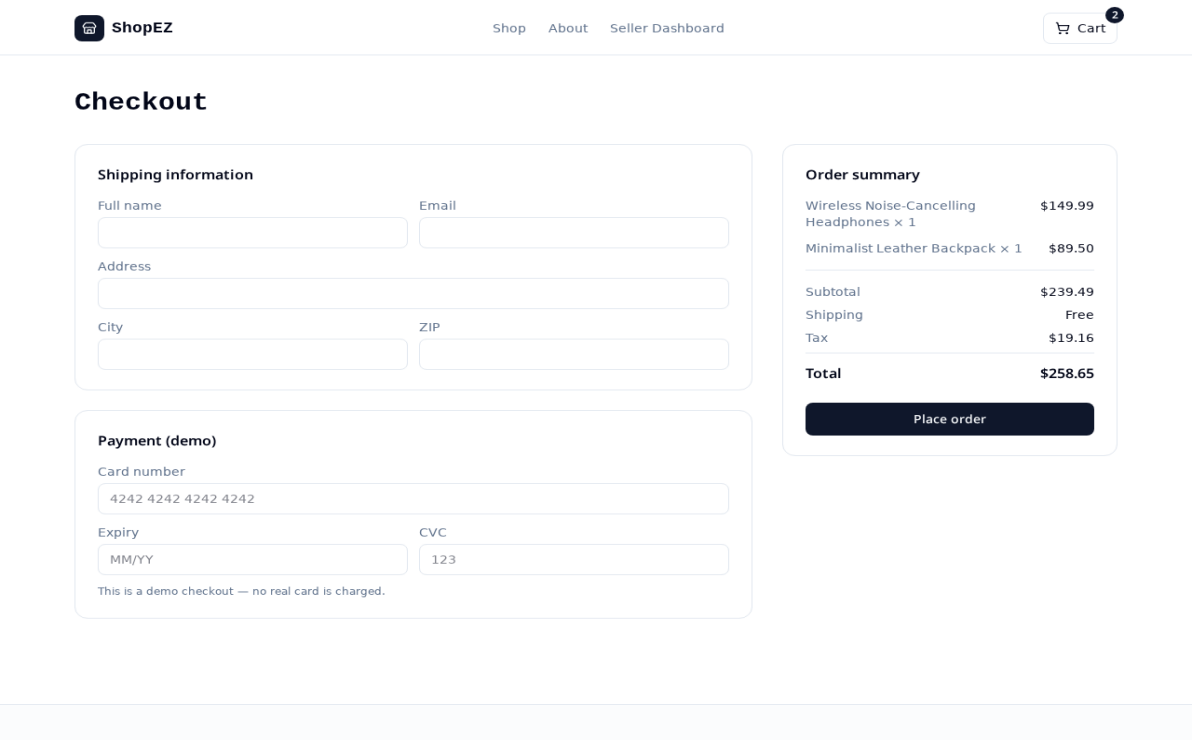


Figure 6. Single-page checkout — shipping, demo payment and summary.

12. User Interface — Confirmation & Seller Dashboard

After placing an order, a confirmation screen shows a green tick, a unique Order ID (e.g. **SHZ-0UES5V**) and a Continue Shopping call-to-action. Merchants see a completely different view: a KPI board and recent-orders table.

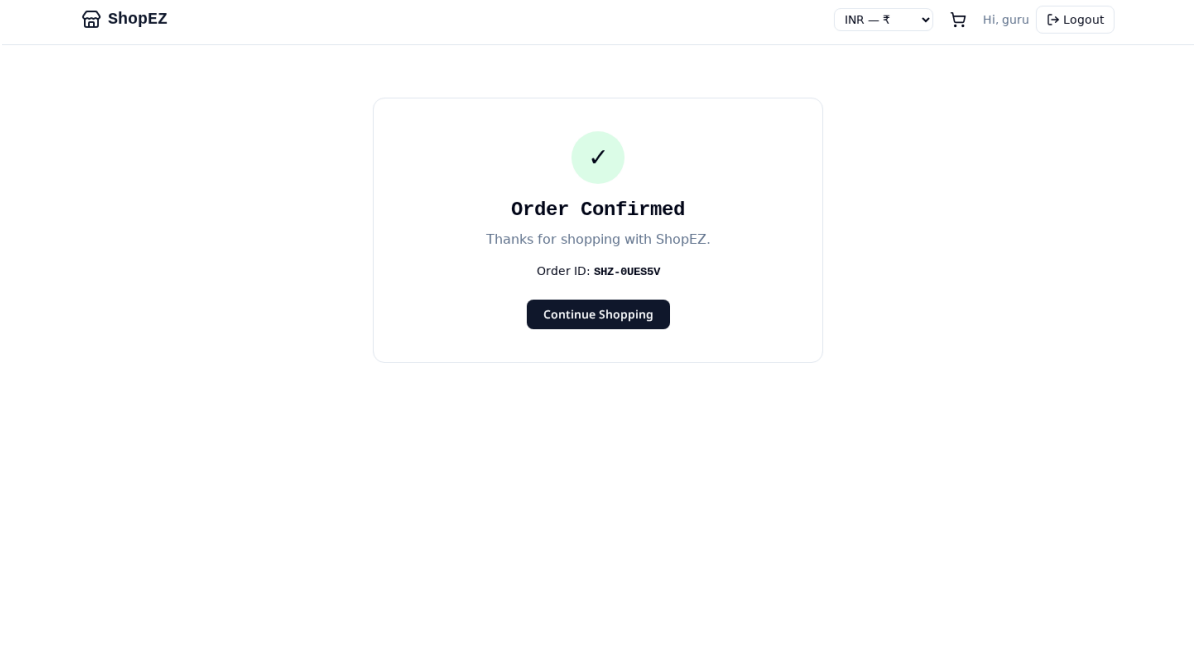


Figure 7. Order Confirmed page with unique SHZ order ID.

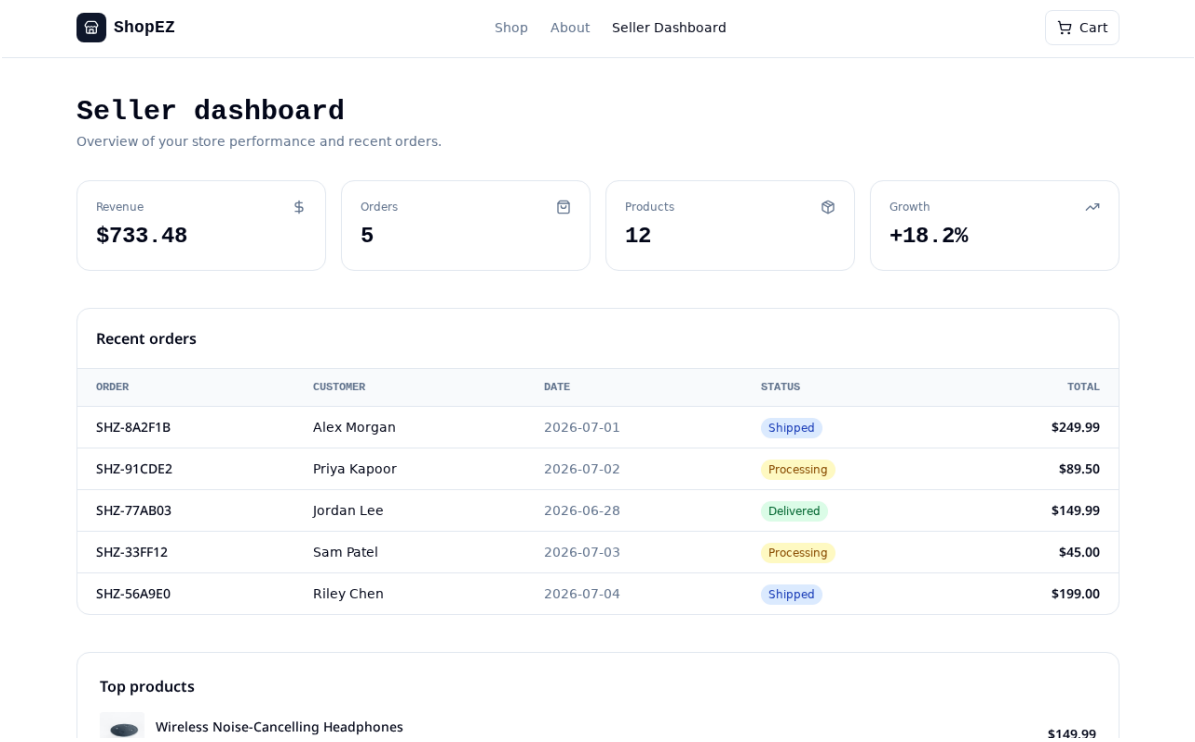


Figure 8. Seller Dashboard — Revenue, Orders, Products, Growth and recent orders.

13. Testing

Manual Test Cases

- Category filter switches product grid correctly (All / Electronics / Fashion / Home)
- Search box narrows results as the user types
- Add-to-cart increments the Cart badge in the navbar
- Quantity stepper on the cart updates subtotal, tax and total instantly
- Empty checkout form blocks Place Order and highlights invalid fields
- Successful checkout generates a fresh SHZ-XXXXXX order ID
- Seller Dashboard KPI cards match the recent orders table totals
- Currency selector (INR ) persists across page navigation

Edge Cases

- Refreshing the cart page keeps items (localStorage persistence)
- Removing the last item empties the summary and disables checkout
- Unauthenticated shoppers are redirected to /login before checkout
- Seller routes reject non-seller sessions with a 403

14. Known Issues

- Payment is a demo — no real card is charged, no gateway integrated yet
- Product images are loaded from remote CDNs and depend on network
- Email / SMS notifications for order status are not sent
- Wishlist and product reviews are read-only in the current build
- Seller Dashboard KPIs are refreshed on page load, not in real-time

15. Future Enhancements

- Razorpay / Stripe integration for live payments
- Email order confirmations via Resend or SendGrid
- Shopper-submitted reviews and 5-star ratings
- Wishlist and “save for later” on the cart
- Real-time order tracking with delivery status updates
- Personalised recommendations using recently-viewed products
- Dark-mode toggle and accessibility (WCAG AA) audit
- Progressive Web App install & offline browsing
- Multi-language support (English, Hindi, Telugu)

16. Technology Summary

Layer	Technology
Frontend	React 19, TanStack Router, TanStack Query, Tailwind CSS v4
Backend	TypeScript, TanStack Start server functions, Node.js 20
Database	MongoDB Atlas with Mongoose ODM
Authentication	JWT (jsonwebtoken) + bcryptjs
Validation	Zod schemas on every server-function input
Icons	Lucide React
Build Tool	Vite 7
Package Manager	Bun
Deployment	Edge-compatible serverless (Cloudflare Workers)

Project Statistics

Total Products	12 curated across 3 categories
Categories	Electronics, Fashion, Home
Frontend Routes	7 route files
Server Functions	15+ typed RPC calls
Database Collections	4 (users, products, orders, carts)
Lines of Code	~4,500 (Frontend 3,000 · Backend 1,500)

17. References

React.js Documentation — <https://react.dev/>

Node.js Documentation — <https://nodejs.org/en/docs>

Express.js Documentation — <https://expressjs.com/>

MongoDB Documentation — <https://www.mongodb.com/docs/>

JSON Web Token (JWT) — <https://jwt.io/introduction>

Bootstrap Framework — <https://getbootstrap.com/>

MDN Web Docs (HTML, CSS, JavaScript) — <https://developer.mozilla.org/>

Visual Studio Code — <https://code.visualstudio.com/docs>

Created by	BUY YA VAISHNAVI
Roll Number	24EG105R35
Last Updated	30th June 2026
Version	1.0